Atividade 08 – Hamming (31, 26)

Nome: Matheus Milanezi

Disciplina: Redes de Computadores: Internetworking, Roteamento e Transmissão

Linguagem: Python 3.14 (arquivo único `hamming.py`, sem bibliotecas externas)

1. Objetivo

Implementar e validar a codificação Hamming (31, 26): receber 26 bits de dados, gerar a palavra de 31 bits com 5 bits de paridade e, na recepção, calcular a síndrome para localizar e corrigir um erro em um único bit.

2. Implementação

Convenções adotadas

- As posições são numeradas de **1 a 31 da esquerda para a direita**: o primeiro caractere da palavra é a posição 1.
- Os bits de paridade ficam nas posições **1, 2, 4, 8 e 16** (potências de 2).
- Os 26 bits de dados (D1 a D26) ocupam as demais posições, em ordem crescente.
- É usada **paridade par**: cada grupo (o bit de paridade e as posições que ele verifica) deve ter uma quantidade par de bits 1.

A convenção precisa ser fixada porque a literatura usa as duas formas (posição 1 à esquerda ou à direita), e uma mesma entrada gera palavras diferentes conforme a escolha.

Mapeamento dos dados

Dado	D1	D2	D3	D4	D5	D6	D7	D8	D9	D10	D11	D12	D13
Posição	3	5	6	7	9	10	11	12	13	14	15	17	18

Dado	D14	D15	D16	D17	D18	D19	D20	D21	D22	D23	D24	D25	D26
Posição	19	20	21	22	23	24	25	26	27	28	29	30	31

Regra de cobertura

O bit de paridade da posição *p* verifica todas as posições *i* cujo número em binário tem o bit de *p* ligado (no código, `i & p != 0`):

Paridade	Bit do índice	Posições verificadas
P1	bit 0 (…xxxx**1**)	1, 3, 5, 7, 9, 11, 13, 15, 17, 19, 21, 23, 25, 27, 29, 31
P2	bit 1 (…xxx**1**x)	2, 3, 6, 7, 10, 11, 14, 15, 18, 19, 22, 23, 26, 27, 30, 31
P4	bit 2 (…xx**1**xx)	4, 5, 6, 7, 12, 13, 14, 15, 20, 21, 22, 23, 28, 29, 30, 31
P8	bit 3 (…x**1**xxx)	8 a 15 e 24 a 31
P16	bit 4 (**1**xxxx)	16 a 31

Cada grupo tem 16 posições: o próprio bit de paridade e 15 bits de dados.

Estrutura do programa

Função	O que faz
`validar(bits, tamanho)`	Remove espaços e rejeita entradas com tamanho diferente do esperado ou com caracteres que não sejam 0 e 1
`cobertas(p)`	Lista as posições verificadas pela paridade *p*
`codificar(dados)`	Coloca D1..D26 nas posições de dados e calcula P1, P2, P4, P8 e P16 com paridade par
`sindrome(palavra)`	Recalcula os 5 grupos na palavra recebida; cada grupo que falha soma o seu peso (1, 2, 4, 8, 16) à síndrome
`corrigir(palavra)`	Se a síndrome for diferente de 0, inverte o bit da posição indicada por ela
`extrair_dados(palavra)`	Recupera os 26 bits de dados da palavra

Internamente, a palavra é montada numa lista de 32 elementos em que o índice 0 não é usado, de modo que `palavra[i]` corresponde exatamente à posição *i* do Hamming. Nas strings, que começam no índice 0, a posição *i* é acessada como `palavra[i - 1]`.

Formas de execução:

```
python hamming.py                          demonstração completa (3 testes + erro + autoteste)
python hamming.py -e BITS26 -e BITS26 ...  demonstração com outras entradas
python hamming.py --erro 16                escolhe a posição do bit a inverter
python hamming.py --codificar BITS26       codifica uma entrada e detalha as paridades
python hamming.py --verificar BITS31       calcula a síndrome e corrige uma palavra recebida
python hamming.py --interativo             lê a entrada pelo teclado
```

O código-fonte completo está no **Apêndice A**.

3. Validação da implementação

Os três testes

Saída de `python hamming.py`:

Teste	Entrada (26 bits)	Palavra Hamming (31, 26)	P1 P2 P4 P8 P16
1	10101010101010101010101010	1011010010101011010101010101010	1 0 1 0 1
2	11111111111111111111111111	1111111111111111111111111111111	1 1 1 1 1
3	11010011100101110001011010	0010101100111000101110001011010	0 0 0 1 0

O programa também exibe cada palavra alinhada com o número da posição e o tipo de bit (P = paridade, d = dado). Teste 3:

```
           0000000001111111111222222222233
 posição   1234567890123456789012345678901
 tipo      PPdPdddPddddddddPddddddddddddddd
 palavra   0010101100111000101110001011010
```

Evidência: print da execução de `python hamming.py` (Partes 1 e 2).

```
======================================================================
PARTES 1 e 2 - Codificação Hamming (31, 26)
======================================================================
Mapeamento dos bits de dados nas posições da palavra:
  D1->3   D2->5   D3->6   D4->7   D5->9   D6->10  D7->11  D8->12  D9->13
  D10->14 D11->15 D12->17 D13->18 D14->19 D15->20 D16->21 D17->22 D18->23
  D19->24 D20->25 D21->26 D22->27 D23->28 D24->29 D25->30 D26->31

Teste 1
  entrada  (26 bits): 10101010101010101010101010
  Hamming  (31 bits): 1011010010101011010101010101010
  agrupado de 4 em 4: 1011 0100 1010 1011 0101 0101 0101 010
  paridades P1 P2 P4 P8 P16 = 1 0 1 0 1
            0000000001111111111222222222233
  posição   1234567890123456789012345678901
  tipo      PPdPdddPddddddddPddddddddddddddd
  palavra   1011010010101011010101010101010

Teste 2
  entrada  (26 bits): 11111111111111111111111111
  Hamming  (31 bits): 1111111111111111111111111111111
  agrupado de 4 em 4: 1111 1111 1111 1111 1111 1111 1111 111
  paridades P1 P2 P4 P8 P16 = 1 1 1 1 1
            0000000001111111111222222222233
  posição   1234567890123456789012345678901
  tipo      PPdPdddPddddddddPddddddddddddddd
  palavra   1111111111111111111111111111111

Teste 3
  entrada  (26 bits): 11010011100101110001011010
  Hamming  (31 bits): 0010101100111000101110001011010
  agrupado de 4 em 4: 0010 1011 0011 1000 1011 1000 1011 010
  paridades P1 P2 P4 P8 P16 = 0 0 0 1 0
            0000000001111111111222222222233
  posição   1234567890123456789012345678901
  tipo      PPdPdddPddddddddPddddddddddddddd
  palavra   0010101100111000101110001011010
```

O Teste 2 tem um resultado que pode ser previsto sem o programa: cada paridade verifica 15 bits de dados, e com todos os dados em 1 cada grupo tem 15 bits em 1, uma quantidade ímpar. Para que o total do grupo fique par, todas as paridades precisam valer 1, e a palavra inteira fica com 31 bits em 1. A previsão foi feita antes da execução e confirmada pela saída do programa.

Como os cinco bits de paridade foram obtidos (Teste 3)

Para cada paridade, contam-se os bits 1 entre os 15 bits de dados que ela verifica. Se a quantidade for ímpar, a paridade vale 1, para tornar o grupo par. Se for par, vale 0.

Paridade	Bits de dados verificados (posição=valor)	Qtde. de 1s	Valor
P1	3=1 5=1 7=1 9=0 11=1 13=1 15=0 17=1 19=1 21=1 23=0 25=1 27=1 29=0 31=0	10 (par)	**0**
P2	3=1 6=0 7=1 10=0 11=1 14=0 15=0 18=0 19=1 22=0 23=0 26=0 27=1 30=1 31=0	6 (par)	**0**
P4	5=1 6=0 7=1 12=1 13=1 14=0 15=0 20=1 21=1 22=0 23=0 28=1 29=0 30=1 31=0	8 (par)	**0**

Paridade	Bits de dados verificados (posição=valor)	Qtde. de 1s	Valor
P8	9=0 10=0 11=1 12=1 13=1 14=0 15=0 24=0 25=1 26=0 27=1 28=1 29=0 30=1 31=0	7 (ímpar)	**1**
P16	17=1 18=0 19=1 20=1 21=1 22=0 23=0 24=0 25=1 26=0 27=1 28=1 29=0 30=1 31=0	8 (par)	**0**

Resultado: P1 P2 P4 P8 P16 = 0 0 0 1 0, que são exatamente os bits das posições 1, 2, 4, 8 e 16 da palavra `00101011001110001011100010110 10`.

O cálculo de P8 foi conferido manualmente: nas posições 9 a 15 os dados são 0 0 1 1 1 0 0 (três bits em 1) e nas posições 24 a 31 são 0 1 0 1 1 0 1 0 (quatro bits em 1). O total é 7, ímpar, então P8 = 1, igual ao calculado pelo programa.

Evidência: print do trecho "Como os 5 bits de paridade foram obtidos no Teste 3".

```
Como os 5 bits de paridade foram obtidos no Teste 3:
  P1  (posição  1, bit 0 do índice) verifica as posições:
       1 3 5 7 9 11 13 15 17 19 21 23 25 27 29 31
       bits de dados verificados: 3=1 5=1 7=1 9=0 11=1 13=1 15=0 17=1 19=1 21=1 23=0 25=1 27=1 29=0 31=0
       quantidade de 1s = 10 (par) -> P1 = 0
  P2  (posição  2, bit 1 do índice) verifica as posições:
       2 3 6 7 10 11 14 15 18 19 22 23 26 27 30 31
       bits de dados verificados: 3=1 6=0 7=1 10=0 11=1 14=0 15=0 18=0 19=1 22=0 23=0 26=0 27=1 30=1 31=0
       quantidade de 1s = 6 (par) -> P2 = 0
  P4  (posição  4, bit 2 do índice) verifica as posições:
       4 5 6 7 12 13 14 15 20 21 22 23 28 29 30 31
       bits de dados verificados: 5=1 6=0 7=1 12=1 13=1 14=0 15=0 20=1 21=1 22=0 23=0 28=1 29=0 30=1 31=0
       quantidade de 1s = 8 (par) -> P4 = 0
  P8  (posição  8, bit 3 do índice) verifica as posições:
       8 9 10 11 12 13 14 15 24 25 26 27 28 29 30 31
       bits de dados verificados: 9=0 10=0 11=1 12=1 13=1 14=0 15=0 24=0 25=1 26=0 27=1 28=1 29=0 30=1 31=0
       quantidade de 1s = 7 (ímpar) -> P8 = 1
  P16 (posição 16, bit 4 do índice) verifica as posições:
       16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31
       bits de dados verificados: 17=1 18=0 19=1 20=1 21=1 22=0 23=0 24=0 25=1 26=0 27=1 28=1 29=0 30=1 31=0
       quantidade de 1s = 8 (par) -> P16 = 0
```

Validação da entrada

O programa rejeita entradas que não tenham exatamente 26 bits ou que contenham caracteres diferentes de 0 e 1. Espaços são ignorados, para permitir digitar os bits em grupos.

Evidência: print das três execuções com entrada inválida.

```
PS D:\Servidor\Faculdade\UNISINOS\2026 2\redes de computadores\Atividade 8> python hamming.py --codificar "1010"
  Erro: esperados 26 bits, recebidos 4
PS D:\Servidor\Faculdade\UNISINOS\2026 2\redes de computadores\Atividade 8> python hamming.py --codificar "1010 1010 1010 1010 1010 1x"
  Erro: a entrada deve conter apenas 0 e 1 (encontrado: x)
PS D:\Servidor\Faculdade\UNISINOS\2026 2\redes de computadores\Atividade 8> python hamming.py --codificar "10101010101010101010101010101"
  Erro: esperados 26 bits, recebidos 27
PS D:\Servidor\Faculdade\UNISINOS\2026 2\redes de computadores\Atividade 8>
```

4. Teste de detecção e correção de erro

Erro em um bit de dados (posição 13)

Na palavra do Teste 3 foi invertido o bit da posição 13, que contém o dado D9 (1 → 0):

	Palavra
Correta	`00101011001110001011100010110 10`
Com erro (posição 13)	`00101011001100001011100010110 10`

Na recepção, o programa recalcula os cinco grupos de paridade sobre a palavra recebida, agora incluindo o próprio bit de paridade:

Verificação	Qtde. de 1s no grupo	Resultado
C1	9	1 – **falhou**
C2	6	0 – ok
C4	7	1 – **falhou**
C8	7	1 – **falhou**
C16	8	0 – ok

Síndrome (C16 C8 C4 C2 C1) = 01101_2 = **13**, que é exatamente a posição alterada.

Posição identificada	13 (D9)
Palavra corrigida	`00101011001110001011100010 11010`
Igual à original?	Sim
Síndrome após a correção	0
Dados recuperados	`11010011100101110001011010` (iguais à entrada do Teste 3)

Evidência: print da Parte 3 da execução de `python hamming.py`.

```
======================================================================
PARTE 3 - Erro de um único bit (palavra do Teste 3)
======================================================================
Palavra correta     : 00101011001110001011100010 11010
Bit invertido       : posição 13 (D9), 1 -> 0
Palavra com erro    : 00101011001100001011100010 11010

            00000000011111111112222222222233
  posição   12345678901234567890123456789 01
  tipo      PPdPdddPddddddddPddddddddddddddd P
  correta   00101011001110001011100010 11010
            00000000011111111112222222222233
  posição   12345678901234567890123456789 01
  tipo      PPdPdddPddddddddPddddddddddddddd P
  com erro  00101011001100001011100010 11010
                       ^ posição 13 (D9)

Verificação da palavra recebida:
  C1  = XOR das posições {1,3,5,7,9,11,13,15,17,19,21,23,25,27,29,31}
        quantidade de 1s = 9 -> C1 = 1  [FALHOU]
  C2  = XOR das posições {2,3,6,7,10,11,14,15,18,19,22,23,26,27,30,31}
        quantidade de 1s = 6 -> C2 = 0  [ok]
  C4  = XOR das posições {4,5,6,7,12,13,14,15,20,21,22,23,28,29,30,31}
        quantidade de 1s = 7 -> C4 = 1  [FALHOU]
  C8  = XOR das posições {8,9,10,11,12,13,14,15,24,25,26,27,28,29,30,31}
        quantidade de 1s = 7 -> C8 = 1  [FALHOU]
  C16 = XOR das posições {16,17,18,19,20,21,22,23,24,25,26,27,28,29,30,31}
        quantidade de 1s = 8 -> C16 = 0  [ok]
  síndrome (C16 C8 C4 C2 C1) = 01101b = 13

Posição identificada: 13 (D9)
Palavra corrigida   : 00101011001110001011100010 11010
Igual à original?   : SIM
Síndrome após correção: 0
Dados recuperados   : 11010011100101110001011010

======================================================================
AUTOTESTE - todas as posições de erro possíveis
======================================================================
  Teste 1: síndrome da palavra sem erro = 0; erros de 1 bit localizados e corrigidos: 31/31
  Teste 2: síndrome da palavra sem erro = 0; erros de 1 bit localizados e corrigidos: 31/31
  Teste 3: síndrome da palavra sem erro = 0; erros de 1 bit localizados e corrigidos: 31/31
  1000 entradas aleatórias com 1 bit invertido: 1000/1000 corrigidas
  síndrome == XOR dos índices dos bits 1: 1000/1000
```

Erro em um bit de paridade (posição 16)

Para confirmar que o erro também é localizado quando atinge um bit de paridade, foi executado `python hamming.py --erro 16`, que inverte o próprio P16 da mesma palavra (0 → 1). Apenas C16 falhou (9 bits em 1 no grupo), e a síndrome foi 10000_2 = **16**. O programa corrigiu a posição 16 e os

dados recuperados foram os mesmos, já que nenhum bit de dados tinha sido atingido. Esse resultado tinha sido previsto antes da execução: a posição 16 pertence a um único grupo, o do P16.

Evidência: print da execução de `python hamming.py --erro 16` (arquivo `saida_erro_p16.txt`).

```
======================================================================
PARTES 1 e 2 - Codificação Hamming (31, 26)
======================================================================
Mapeamento dos bits de dados nas posições da palavra:
  D1->3   D2->5   D3->6   D4->7   D5->9   D6->10  D7->11  D8->12  D9->13
  D10->14 D11->15 D12->17 D13->18 D14->19 D15->20 D16->21 D17->22 D18->23
  D19->24 D20->25 D21->26 D22->27 D23->28 D24->29 D25->30 D26->31

Teste 1
  entrada  (26 bits): 10101010101010101010101010
  Hamming  (31 bits): 1011010010101011010101010101010
  agrupado de 4 em 4: 1011 0100 1010 1011 0101 0101 0101 010
  paridades P1 P2 P4 P8 P16 = 1 0 1 0 1
        0000000001111111111222222222233
  posição   1234567890123456789012345678901
  tipo      PPdPdddPddddddddPddddddddddddddd
  palavra   1011010010101011010101010101010

Teste 2
  entrada  (26 bits): 11111111111111111111111111
  Hamming  (31 bits): 1111111111111111111111111111111
  agrupado de 4 em 4: 1111 1111 1111 1111 1111 1111 1111 111
  paridades P1 P2 P4 P8 P16 = 1 1 1 1 1
        0000000001111111111222222222233
  posição   1234567890123456789012345678901
  tipo      PPdPdddPddddddddPddddddddddddddd
  palavra   1111111111111111111111111111111

Teste 3
  entrada  (26 bits): 11010011100101110001011010
```

Hamming (31 bits): 0010101100111000101110001011010

agrupado de 4 em 4: 0010 1011 0011 1000 1011 1000 1011 010

paridades P1 P2 P4 P8 P16 = 0 0 0 1 0

0000000001111111111222222222233

posição 1234567890123456789012345678901

tipo PPdPdddPddddddddPddddddddddddddd

palavra 0010101100111000101110001011010

Como os 5 bits de paridade foram obtidos no Teste 3:

P1 (posição 1, bit 0 do índice) verifica as posições:

1 3 5 7 9 11 13 15 17 19 21 23 25 27 29 31

bits de dados verificados: 3=1 5=1 7=1 9=0 11=1 13=1 15=0 17=1 19=1 21=1 23=0 25=1 27=1 29=0 31=0

quantidade de 1s = 10 (par) -> P1 = 0

P2 (posição 2, bit 1 do índice) verifica as posições:

2 3 6 7 10 11 14 15 18 19 22 23 26 27 30 31

bits de dados verificados: 3=1 6=0 7=1 10=0 11=1 14=0 15=0 18=0 19=1 22=0 23=0 26=0 27=1 30=1 31=0

quantidade de 1s = 6 (par) -> P2 = 0

P4 (posição 4, bit 2 do índice) verifica as posições:

4 5 6 7 12 13 14 15 20 21 22 23 28 29 30 31

bits de dados verificados: 5=1 6=0 7=1 12=1 13=1 14=0 15=0 20=1 21=1 22=0 23=0 28=1 29=0 30=1 31=0

quantidade de 1s = 8 (par) -> P4 = 0

P8 (posição 8, bit 3 do índice) verifica as posições:

8 9 10 11 12 13 14 15 24 25 26 27 28 29 30 31

bits de dados verificados: 9=0 10=0 11=1 12=1 13=1 14=0 15=0 24=0 25=1 26=0 27=1 28=1 29=0 30=1 31=0

quantidade de 1s = 7 (ímpar) -> P8 = 1

P16 (posição 16, bit 4 do índice) verifica as posições:

16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31

bits de dados verificados: 17=1 18=0 19=1 20=1 21=1 22=0 23=0 24=0 25=1 26=0 27=1 28=1 29=0 30=1 31=0

quantidade de 1s = 8 (par) -> P16 = 0

```
================================================================================
PARTE 3 - Erro de um único bit (palavra do Teste 3)
================================================================================
Palavra correta     : 00101011001110001011100010 11010
Bit invertido       : posição 16 (P16), 0 -> 1
Palavra com erro    : 00101011001110011011100010 11010

          00000000011111111111222222222233
  posição   12345678901234567890123456789 01
  tipo      PPdPdddPddddddddPddddddddddddddd
  correta   00101011001110001011100010 11010
          00000000011111111111222222222233
  posição   12345678901234567890123456789 01
  tipo      PPdPdddPddddddddPddddddddddddddd
  com erro  00101011001110011011100010 11010
                      ^ posição 16 (P16)

Verificação da palavra recebida:
  C1  = XOR das posições {1,3,5,7,9,11,13,15,17,19,21,23,25,27,29,31}
        quantidade de 1s = 10 -> C1 = 0  [ok]
  C2  = XOR das posições {2,3,6,7,10,11,14,15,18,19,22,23,26,27,30,31}
        quantidade de 1s = 6 -> C2 = 0  [ok]
  C4  = XOR das posições {4,5,6,7,12,13,14,15,20,21,22,23,28,29,30,31}
        quantidade de 1s = 8 -> C4 = 0  [ok]
  C8  = XOR das posições {8,9,10,11,12,13,14,15,24,25,26,27,28,29,30,31}
        quantidade de 1s = 8 -> C8 = 0  [ok]
  C16 = XOR das posições {16,17,18,19,20,21,22,23,24,25,26,27,28,29,30,31}
        quantidade de 1s = 9 -> C16 = 1  [FALHOU]
  síndrome (C16 C8 C4 C2 C1) = 10000b = 16

Posição identificada: 16 (P16)
```

Palavra corrigida : 0010101100111000101110001011010

Igual à original? : SIM

Síndrome após correção: 0

Dados recuperados : 1101001110010111000101101O

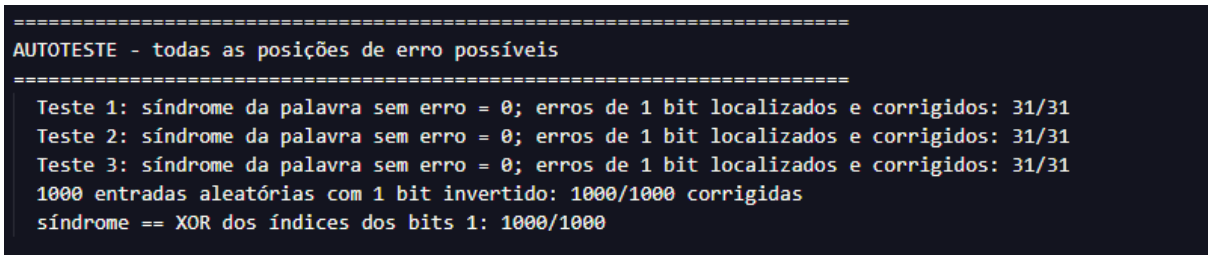
Autoteste: todas as posições

Além dos casos escolhidos acima, o programa inverte, uma de cada vez, cada uma das 31 posições das três palavras, e também testa 1000 entradas aleatórias com um bit invertido em posição aleatória:

```
Teste 1: síndrome da palavra sem erro = 0; erros de 1 bit localizados e corrigidos: 31/31
Teste 2: síndrome da palavra sem erro = 0; erros de 1 bit localizados e corrigidos: 31/31
Teste 3: síndrome da palavra sem erro = 0; erros de 1 bit localizados e corrigidos: 31/31
1000 entradas aleatórias com 1 bit invertido: 1000/1000 corrigidas
síndrome == XOR dos índices dos bits 1: 1000/1000
```

A última linha é uma conferência feita por outro caminho: em vez de recalcular os cinco grupos, faz o XOR dos números de todas as posições que valem 1 na palavra recebida. Os dois métodos deram o mesmo resultado nos 1000 casos.

Evidência: print da seção "AUTOTESTE" da execução.



5. Análise dos resultados

Por que o Hamming (31, 26) utiliza cinco bits de paridade?

Porque a síndrome precisa conseguir representar todas as situações possíveis na recepção: nenhum erro ou um erro em qualquer uma das 31 posições. São 32 situações, e 5 bits representam exatamente $2^5 = 32$ valores. Na forma geral, com *k* bits de dados e *r* de paridade, é preciso que $2^r \geq k + r + 1$. Com r = 5: $32 \geq 26 + 5 + 1 = 32$, o que é satisfeito. Com r = 4: $16 \geq 26 + 4 + 1 = 31$, o que não é.

Como a igualdade é exata, nenhum valor de síndrome fica sem uso, e por isso o (31, 26) é chamado de código **perfeito**. O autoteste confirma isso: nos três testes, cada uma das 31 posições invertidas gerou uma síndrome diferente, de 1 a 31, e a palavra sem erro gerou síndrome 0.

Qual é a relação entre as posições 1, 2, 4, 8 e 16 e o cálculo da síndrome?

Cada uma dessas posições é uma potência de 2, ou seja, tem um único bit ligado em binário (1 = 00001, 2 = 00010, 4 = 00100, 8 = 01000, 16 = 10000). A paridade da posição 2^j verifica todas as posições que têm o bit *j* ligado, e o resultado da sua verificação na recepção vira o bit *j* da síndrome.

Cada bit de paridade é, portanto, o "peso" de um dígito binário da síndrome: C1 vale 1, C2 vale 2, C4 vale 4, C8 vale 8 e C16 vale 16.

Colocar as paridades nessas posições também faz com que cada uma pertença a um único grupo, o seu. Assim cada paridade pode ser calculada só a partir dos dados, sem depender das outras. No teste com erro no P16, isso apareceu na prática: só C16 falhou.

Como a síndrome permite localizar um erro em um único bit?

Uma posição *i* pertence exatamente aos grupos correspondentes aos bits ligados no número *i* em binário. Quando o bit da posição *i* é invertido, a quantidade de 1s muda em exatamente esses grupos, que passam a falhar, e somente neles. A combinação dos grupos que falham é, portanto, o próprio número *i* escrito em binário.

No teste realizado, a posição 13 = 01101_2 pertence aos grupos C1, C4 e C8, que foram exatamente os que falharam, e a síndrome deu 13. No teste com P16, a posição 16 = 10000_2 pertence só ao grupo C16, e a síndrome deu 16. Basta inverter o bit da posição indicada para recuperar a palavra original.

Essa localização só funciona para **um** erro. Foi feito um teste extra invertendo as posições 3 e 5 da mesma palavra: a síndrome deu 6, porque 3 XOR 5 = 011 XOR 101 = 110. O programa "corrigiu" a posição 6, que estava certa, e a palavra final ficou com 3 bits errados (posições 3, 5 e 6). Com dois erros, a síndrome aponta para uma terceira posição e a correção piora a palavra. A verificação mostra o motivo: as posições 3 (00011) e 5 (00101) pertencem ambas ao grupo do P1, e as duas inversões se anularam nesse grupo, que continuou com total par (C1 passou). Já nos grupos C2 e C4, cada um contendo só uma das posições alteradas, a verificação falhou, formando 00110 = 6. Para pelo menos detectar erros duplos seria necessário um bit de paridade global adicional (esquema SECDED, que formaria um código (32, 26)).

Evidência: print de `python hamming.py --verificar 00000011001110001011110001011010` (palavra do Teste 3 com as posições 3 e 5 invertidas) e arquivo `saida_erro_duplo.txt`.

```
======================================================================
Verificação de palavra recebida
======================================================================
            00000000011111111112222222222233
  posição   12345678901234567890123456789012
  tipo      PPdPdddPddddddddPddddddddddddddd
  recebida  00000011001110001011110001011010

  C1  = XOR das posições {1,3,5,7,9,11,13,15,17,19,21,23,25,27,29,31}
        quantidade de 1s = 8 -> C1 = 0  [ok]
  C2  = XOR das posições {2,3,6,7,10,11,14,15,18,19,22,23,26,27,30,31}
        quantidade de 1s = 5 -> C2 = 1  [FALHOU]
  C4  = XOR das posições {4,5,6,7,12,13,14,15,20,21,22,23,28,29,30,31}
        quantidade de 1s = 7 -> C4 = 1  [FALHOU]
  C8  = XOR das posições {8,9,10,11,12,13,14,15,24,25,26,27,28,29,30,31}
        quantidade de 1s = 8 -> C8 = 0  [ok]
  C16 = XOR das posições {16,17,18,19,20,21,22,23,24,25,26,27,28,29,30,31}
        quantidade de 1s = 8 -> C16 = 0  [ok]
  síndrome (C16 C8 C4 C2 C1) = 00110b = 6
Erro na posição 6 (D3)
Palavra corrigida: 00000111001110001011110001011010
Dados: 00110011100101110001011010
```

Qual é o overhead introduzido pela codificação Hamming (31, 26)?

São 5 bits de paridade para cada 26 bits de dados.

- Em relação à palavra transmitida: 5/31 ≈ **16,1 %** dos bits enviados são redundância.
- Em relação aos dados: 5/26 ≈ **19,2 %** de bits a mais precisam ser transmitidos.
- A taxa do código é 26/31 ≈ **83,9 %**, ou seja, essa é a fração útil de cada palavra.

Para comparação, o Hamming (7, 4), que tem a mesma capacidade de corrigir um erro por palavra, tem overhead de 3/7 ≈ 42,9 %. Palavras maiores diluem o custo das paridades, porque elas crescem de forma logarítmica (r bits protegem até $2^r - 1$ posições). Em troca, cada palavra continua corrigindo só um erro, e por isso um código maior protege menos quando os erros são frequentes.

6. Uso de Inteligência Artificial

Ferramenta utilizada: Claude (Anthropic), via Claude Code.

Etapas em que ajudou: explicação inicial do funcionamento do código (regra de cobertura, condição $2^r \geq k + r + 1$ e síndrome); escrita do programa `hamming.py`, incluindo a validação da entrada, as funções de exibição e o autoteste; escolha dos casos de teste; e previsão dos resultados antes de cada execução.

Como as sugestões foram validadas: cada previsão foi comparada com a saída real do programa. A correção do código como um todo foi validada pelo autoteste, que inverte todas as 31 posições das três palavras e mais 1000 palavras aleatórias, e por uma conferência independente da síndrome (XOR dos índices dos bits 1), que concordou em 1000 de 1000 casos. Além disso, uma das paridades (P8 do Teste 3) foi recalculada manualmente e coincidiu com o valor do programa.

Previsões confirmadas pela execução:

- O Teste 2 (26 bits em 1) geraria uma palavra com 31 bits em 1, porque cada grupo teria 15 bits de dados em 1. Confirmado.
- O erro na posição 13 = 01101_2 faria falhar exatamente C1, C4 e C8, com síndrome 13. Confirmado.
- O erro no próprio P16 faria falhar só C16, com síndrome 16, sem alterar os dados recuperados. Confirmado.

Sugestão que precisou ser confirmada e adaptada: a IA apontou dois pontos em que uma implementação de Hamming costuma errar sem que o erro fique evidente. O primeiro é a **ordem das posições**: a posição 1 pode ser o primeiro ou o último caractere da string. As duas formas existem na literatura, e a escolha muda a palavra gerada. A convenção foi fixada (posição 1 à esquerda) e documentada no código e na seção 2. O segundo é o **deslocamento de índice**: as posições do Hamming começam em 1 e as strings em Python começam em 0. Um erro de uma unidade aqui ainda produziria palavras "parecidas" com as corretas, mas a síndrome apontaria a posição vizinha à do erro. A implementação adotou uma lista com o índice 0 sem uso no codificador e o acesso `palavra[i - 1]` nas strings. A conversão foi confirmada pelo autoteste: se houvesse deslocamento, a síndrome não coincidiria com a posição invertida, e o resultado foi 31/31 nas três palavras.

Uma sugestão da IA também foi corrigida depois da primeira execução: a tabela de mapeamento dos dados (D1→3, D2→5, …) saiu desalinhada a partir de D10, porque a largura da coluna não comportava os nomes com dois dígitos. A largura foi ajustada e a saída foi conferida de novo.

7. Conclusão

O codificador Hamming (31, 26) foi implementado e funcionou em todos os testes: as três entradas geraram palavras de 31 bits cujas paridades podem ser conferidas grupo a grupo, e o erro de um bit foi localizado e corrigido tanto num bit de dados (posição 13) quanto num bit de paridade (posição 16). O autoteste ampliou essa verificação para todas as 31 posições possíveis e para 1000 palavras aleatórias, sem nenhuma falha.

O ponto central do código ficou visível nos resultados: as paridades nas posições potência de 2 transformam a verificação na recepção em um número binário, e esse número é a posição do erro. O mesmo mecanismo explica o limite do código, observado no teste com dois erros: a síndrome passa a ser o XOR das duas posições e aponta para uma terceira, de modo que o Hamming (31, 26), com um overhead de cerca de 16 % da palavra, garante a correção de no máximo um erro por palavra.

Apêndice A – Código-fonte completo (`hamming.py`)

```
"""
Atividade 08 - Hamming (31, 26)
Redes de Computadores - UNISINOS 2026/2 - Matheus Milanezi


Convenções adotadas:
  - posições numeradas de 1 a 31 da ESQUERDA para a DIREITA
    (o 1º caractere da palavra é a posição 1);
  - bits de paridade nas posições 1, 2, 4, 8 e 16 (potências de 2);
  - bits de dados D1..D26 ocupam as demais posições, em ordem crescente;
  - paridade PAR: cada grupo (paridade + posições que ela verifica)
    deve ter uma quantidade par de bits 1.


Uso:
  python hamming.py                          demonstração completa (3 testes + erro)
  python hamming.py -e BITS26 -e BITS26 ...  demonstração com suas próprias entradas
  python hamming.py --detalhar 2 --teste-erro 1 --erro 7
  python hamming.py --codificar BITS26       só codifica uma entrada
  python hamming.py --verificar BITS31       calcula a síndrome e corrige uma palavra recebida
  python hamming.py --interativo             lê a entrada pelo teclado
"""


import argparse
import random
import sys


N = 31                                  # bits da palavra codificada
K = 26                                  # bits de dados
PARIDADES = [1, 2, 4, 8, 16]            # posições dos bits de paridade
POS_DADOS = [i for i in range(1, N + 1) if i not in PARIDADES]


ENTRADAS_PADRAO = [
    "10101010101010101010101010",
    "11111111111111111111111111",
    "11010011100101110001011010",
]



# ---------------------------------------------------------------------------
# Núcleo: validação, codificação, síndrome e correção
# ---------------------------------------------------------------------------


def validar(bits: str, tamanho: int) -> str:
    """Remove espaços e confere se a entrada tem exatamente `tamanho` bits 0/1."""
    s = "".join(bits.split())
    invalidos = sorted({c for c in s if c not in "01"})
    if invalidos:
        raise ValueError(f"a entrada deve conter apenas 0 e 1 (encontrado: {' '.join(invalidos)})")
    if len(s) != tamanho:
        raise ValueError(f"esperados {tamanho} bits, recebidos {len(s)}")
    return s



def cobertas(p: int) -> list[int]:
    """Posições verificadas pela paridade p: todas cujo índice binário tem o bit p ligado."""
    return [i for i in range(1, N + 1) if i & p]



def codificar(dados: str) -> str:
    """Gera a palavra Hamming (31, 26) a partir de 26 bits de dados."""
    dados = validar(dados, K)
    palavra = [0] * (N + 1)             # índice 0 não é usado: palavra[i] = posição i
    for pos, bit in zip(POS_DADOS, dados):
        palavra[pos] = int(bit)
    for p in PARIDADES:
        palavra[p] = sum(palavra[i] for i in cobertas(p) if i != p) % 2
    return "".join(str(b) for b in palavra[1:])



def sindrome(palavra: str) -> int:
    """Recalcula cada grupo de paridade; os grupos que falham formam a síndrome."""
    palavra = validar(palavra, N)
    s = 0
    for p in PARIDADES:
```

```
        if sum(int(palavra[i - 1]) for i in cobertas(p)) % 2 == 1:
            s |= p
    return s



def inverter_bit(palavra: str, pos: int) -> str:
    """Inverte o bit da posição `pos` (1..31)."""
    if not 1 <= pos <= N:
        raise ValueError(f"posição deve estar entre 1 e {N}")
    i = pos - 1
    return palavra[:i] + ("1" if palavra[i] == "0" else "0") + palavra[i + 1:]



def corrigir(palavra: str) -> tuple[str, int]:
    """Retorna (palavra corrigida, síndrome). Síndrome 0 = nenhum erro detectado."""
    s = sindrome(palavra)
    if s == 0:
        return palavra, 0
    return inverter_bit(palavra, s), s



def extrair_dados(palavra: str) -> str:
    return "".join(palavra[p - 1] for p in POS_DADOS)



# ---------------------------------------------------------------------------
# Exibição
# ---------------------------------------------------------------------------

def agrupar(bits: str, n: int = 4) -> str:
    return " ".join(bits[i:i + n] for i in range(0, len(bits), n))



def nome(pos: int) -> str:
    return f"P{pos}" if pos in PARIDADES else f"D{POS_DADOS.index(pos) + 1}"



def tabela(palavra: str, marcar: int | None = None, rotulo: str = "palavra") -> None:
    """Mostra a palavra alinhada com o número de cada posição e o tipo do bit (P/D)."""
    ini = f"  {'':<10}"
    print(ini + "".join(str(i // 10) for i in range(1, N + 1)))
    print(f"  {'posição':<10}" + "".join(str(i % 10) for i in range(1, N + 1)))
    print(f"  {'tipo':<10}" + "".join("P" if i in PARIDADES else "d" for i in range(1, N + 1)))
    print(f"  {rotulo:<10}" + palavra)
    if marcar:
        print(ini + " " * (marcar - 1) + f"^ posição {marcar} ({nome(marcar)})")



def mostrar_mapeamento() -> None:
    print("Mapeamento dos bits de dados nas posições da palavra:")
    pares = [f"D{k + 1}->{p}" for k, p in enumerate(POS_DADOS)]
    for i in range(0, len(pares), 9):
        print("  " + " ".join(f"{x:<7}" for x in pares[i:i + 9]))
    print()



def detalhar_paridades(palavra: str) -> None:
    """Mostra como cada paridade foi obtida a partir dos bits de dados que ela verifica."""
    for p in PARIDADES:
        outras = [i for i in cobertas(p) if i != p]
        uns = sum(int(palavra[i - 1]) for i in outras)
        print(f"  P{p:<2} (posição {p:>2}, bit {p.bit_length() - 1} do índice) verifica as posições:")
        print(f"       {' '.join(str(i) for i in cobertas(p))}")
        print(f"       bits de dados verificados: "
              + " ".join(f"{i}={palavra[i - 1]}" for i in outras))
        print(f"       quantidade de 1s = {uns} ({'par' if uns % 2 == 0 else 'ímpar'})"
              f" -> P{p} = {uns % 2}")
    print()



def detalhar_sindrome(palavra: str) -> int:
    """Mostra cada verificação de paridade na palavra recebida e monta a síndrome."""
    bits_s = {}
```

```
    for p in PARIDADES:
        uns = sum(int(palavra[i - 1]) for i in cobertas(p))
        bits_s[p] = uns % 2
        estado = "FALHOU" if uns % 2 else "ok"
        print(f"  C{p:<2} = XOR das posições {{{','.join(str(i) for i in cobertas(p))}}}")
        print(f"        quantidade de 1s = {uns} -> C{p} = {uns % 2}  [{estado}]")
    s = sum(p for p in PARIDADES if bits_s[p])
    binario = "".join(str(bits_s[p]) for p in reversed(PARIDADES))
    print(f"  síndrome (C16 C8 C4 C2 C1) = {binario}b = {s}")
    return s



# ---------------------------------------------------------------------------
# Roteiros da atividade
# ---------------------------------------------------------------------------


def linha(titulo: str) -> None:
    print("=" * 72)
    print(titulo)
    print("=" * 72)



def parte_codificacao(entradas: list[str], detalhar: int) -> list[str]:
    linha("PARTES 1 e 2 - Codificação Hamming (31, 26)")
    mostrar_mapeamento()
    palavras = []
    for n, dados in enumerate(entradas, 1):
        palavra = codificar(dados)
        palavras.append(palavra)
        print(f"Teste {n}")
        print(f"  entrada  (26 bits): {dados}")
        print(f"  Hamming  (31 bits): {palavra}")
        print(f"  agrupado de 4 em 4: {agrupar(palavra)}")
        print(f"  paridades P1 P2 P4 P8 P16 = "
              + " ".join(palavra[p - 1] for p in PARIDADES))
        tabela(palavra)
        print()
    print(f"Como os 5 bits de paridade foram obtidos no Teste {detalhar}:")
    detalhar_paridades(palavras[detalhar - 1])
    return palavras



def parte_erro(palavra: str, pos: int, n_teste: int) -> None:
    linha(f"PARTE 3 - Erro de um único bit (palavra do Teste {n_teste})")
    recebida = inverter_bit(palavra, pos)
    print(f"Palavra correta     : {palavra}")
    print(f"Bit invertido       : posição {pos} ({nome(pos)}), "
          f"{palavra[pos - 1]} -> {recebida[pos - 1]}")
    print(f"Palavra com erro    : {recebida}")
    print()
    tabela(palavra, rotulo="correta")
    tabela(recebida, marcar=pos, rotulo="com erro")
    print()
    print("Verificação da palavra recebida:")
    s = detalhar_sindrome(recebida)
    print()
    corrigida, _ = corrigir(recebida)
    print(f"Posição identificada: {s} ({nome(s)})" if s else "Nenhum erro detectado")
    print(f"Palavra corrigida   : {corrigida}")
    print(f"Igual à original?   : {'SIM' if corrigida == palavra else 'NÃO'}")
    print(f"Síndrome após correção: {sindrome(corrigida)}")
    print(f"Dados recuperados   : {extrair_dados(corrigida)}")
    print()



def autoteste(palavras: list[str], aleatorios: int = 1000) -> None:
    linha("AUTOTESTE - todas as posições de erro possíveis")
    for n, palavra in enumerate(palavras, 1):
        acertos = sum(
            corrigir(inverter_bit(palavra, pos)) == (palavra, pos) for pos in range(1, N + 1)
        )
        print(f"  Teste {n}: síndrome da palavra sem erro = {sindrome(palavra)}; "
              f"erros de 1 bit localizados e corrigidos: {acertos}/{N}")
    rng = random.Random(2026)
    ok = xor_ok = 0
    for _ in range(aleatorios):
        dados = "".join(rng.choice("01") for _ in range(K))
        palavra = codificar(dados)
        pos = rng.randint(1, N)
```

```
        recebida = inverter_bit(palavra, pos)
        corrigida, s = corrigir(recebida)
        ok += (s == pos and corrigida == palavra and extrair_dados(corrigida) == dados)
        # conferência independente: a síndrome é o XOR dos índices das posições com bit 1
        xor_indices = 0
        for i, b in enumerate(recebida, 1):
            if b == "1":
                xor_indices ^= i
        xor_ok += (xor_indices == s)
    print(f"  {aleatorios} entradas aleatórias com 1 bit invertido: {ok}/{aleatorios} corrigidas")
    print(f"  síndrome == XOR dos índices dos bits 1: {xor_ok}/{aleatorios}")
    print()



def modo_verificar(palavra: str) -> None:
    palavra = validar(palavra, N)
    linha("Verificação de palavra recebida")
    tabela(palavra, rotulo="recebida")
    print()
    s = detalhar_sindrome(palavra)
    corrigida, _ = corrigir(palavra)
    if s:
        print(f"Erro na posição {s} ({nome(s)})")
        print(f"Palavra corrigida: {corrigida}")
    else:
        print("Síndrome 0: nenhum erro de bit único detectado")
    print(f"Dados: {extrair_dados(corrigida)}")



def ler_teclado() -> str:
    while True:
        try:
            return validar(input(f"Digite {K} bits de dados: "), K)
        except ValueError as e:
            print(f"  entrada inválida: {e}")



def main() -> None:
    try:
        sys.stdout.reconfigure(encoding="utf-8")
    except (AttributeError, ValueError):
        pass

    ap = argparse.ArgumentParser(description="Codificador/decodificador Hamming (31, 26)")
    ap.add_argument("-e", "--entrada", action="append", metavar="BITS26",
                    help="entrada de 26 bits (repita para vários testes)")
    ap.add_argument("--detalhar", type=int, default=3,
                    help="teste cujas paridades serão detalhadas (padrão: 3)")
    ap.add_argument("--teste-erro", type=int, default=3,
                    help="teste usado na injeção de erro (padrão: 3)")
    ap.add_argument("--erro", type=int, default=13,
                    help="posição (1..31) do bit a inverter (padrão: 13)")
    ap.add_argument("--codificar", metavar="BITS26", help="apenas codifica a entrada")
    ap.add_argument("--verificar", metavar="BITS31", help="verifica/corrige uma palavra recebida")
    ap.add_argument("--interativo", action="store_true", help="lê a entrada pelo teclado")
    args = ap.parse_args()

    try:
        if args.codificar:
            dados = validar(args.codificar, K)
            palavra = codificar(dados)
            print(f"entrada  (26 bits): {dados}")
            print(f"Hamming  (31 bits): {palavra}")
            tabela(palavra)
            print()
            detalhar_paridades(palavra)
            return
        if args.verificar:
            modo_verificar(args.verificar)
            return

        entradas = [ler_teclado()] if args.interativo else (args.entrada or ENTRADAS_PADRAO)
        entradas = [validar(e, K) for e in entradas]
        detalhar = min(args.detalhar, len(entradas))
        teste_erro = min(args.teste_erro, len(entradas))

        palavras = parte_codificacao(entradas, detalhar)
        parte_erro(palavras[teste_erro - 1], args.erro, teste_erro)
```

```
        autoteste(palavras)
    except ValueError as e:
        print(f"Erro: {e}", file=sys.stderr)
        sys.exit(1)



if __name__ == "__main__":
    main()
```